

Complete OSM Geocoder Implementation Manual

From PBF bytes to browser map, reverse geocoding, forward geocoding, POI search, and local Ollama intent parsing

Final implementation documentation

July 2026

This document explains the current implementation as a technical manual: simple enough to follow, but detailed enough to answer implementation questions about parsing, skipped data, memory layout, indexes, ranking, API endpoints, browser behavior, and local LLM handling.

Contents

1	One-page mental picture	6
2	Input file: what an OSM PBF contains	6
2.1	PBF file	6
2.2	Node drawing	6
2.3	Way drawing	7
2.4	Relation drawing	7
2.5	Tags	7
3	Build and runtime shape	8
3.1	Executable	8
3.2	Main command modes	8
4	Parser: what it keeps	8
4.1	House records	8
4.2	Street records	9
4.3	Admin area records	9
4.4	POI records	9
5	Parser: what it skips	10
5.1	Skipped street-like ways	10
5.2	Unnamed street recovery	10
5.3	Skipped POIs	10
5.4	Skipped raw data	10
6	Low-memory PBF reading	11
6.1	Why multiple scans exist	11
6.2	NeededNodeStore	11

7	Administrative boundary assembly	12
7.1	Relation way spans	12
7.2	Ring building	12
7.3	Inner holes	12
8	Point-in-polygon and admin links	12
8.1	Point-in-polygon test	12
8.2	Admin links for objects	13
9	Street connection	13
9.1	Why streets are connected	13
9.2	How connection works	13
10	Compact dataset memory model	13
10.1	String table	13
10.2	Geometry arrays	13
10.3	Checked 32-bit offsets	14
11	Binary snapshot	14
11.1	What is written	14
11.2	Why the forward index is inside the snapshot	14
11.3	What still builds at server startup	14
12	Server startup	15
12.1	Server constructor	15
12.2	Spatial grid drawing	15
13	HTTP API	15
13.1	Stats endpoint	15
13.2	Viewport endpoints	15
13.3	Search endpoints	15
14	Frontend map	15
14.1	Layer display	15
14.2	Viewport loading control	16
14.3	Result highlighting	16
15	Forward geocode index	16
15.1	Inverted index	16
15.2	Context index	16
15.3	Primary index	17
15.4	Substring suffix array	17
15.5	Fuzzy search	17

16 Forward query matching	17
16.1 Normalization	17
16.2 Candidate intersection	18
17 Forward ranking formulas	18
17.1 House ranking	18
17.2 Street ranking	18
17.3 Admin ranking	18
17.4 POI ranking	19
17.5 Additional candidate helpers	20
18 Reverse geocoder	20
18.1 Input and output	20
18.2 Distance search	20
19 Natural-language and Ollama mode	20
19.1 What Ollama is allowed to do	20
19.2 Two verification passes	21
19.3 Natural address guard	21
19.4 Product and service concept graph	21
19.5 Local knowledge graph in depth	22
20 Frame of reference	25
20.1 Map center	25
20.2 When place names override the current view	25
20.3 When the current view helps	25
21 Browser controls	25
21.1 Geocoder text field	25
21.2 Start Ollama Service button	25
21.3 Reverse geocoder checkbox	25
22 Performance controls	25
22.1 Progress bars	25
22.2 Why ETA can be unknown	26
22.3 Why speed can drop during parsing	26
23 Memory controls	26
23.1 Released string interner index	26
23.2 Flat relation geometry	26
23.3 Moved posting lists	26
23.4 Packed geocode refs	26

24 Exact behavior examples	26
24.1 Kaistrasse 5 Kiel	26
24.2 Stuttgart from Barcelona	27
24.3 Stuttgart Burger King	27
25 Complete implementation walkthrough	27
25.1 PBF primitives in the program	27
25.2 Tag reading and name selection	27
25.3 Address extraction from nodes	28
25.4 Address extraction from building ways	28
25.5 Address extraction from building relations	29
25.6 Street extraction and street skip rules	29
25.7 Unnamed street recovery	29
25.8 Street connection in full detail	29
25.9 POI extraction step by step	30
25.10 Administrative boundary extraction	30
25.11 Inner holes in admin areas	31
25.12 Admin containment and object links	31
25.13 Reverse geocoder step by step	31
25.14 Forward geocode index construction	32
25.15 Forward search query path	32
25.16 House ranking details	33
25.17 Street ranking details	33
25.18 Admin ranking details	33
25.19 POI ranking details	34
25.20 Natural-language search path	34
25.21 Natural address guard	34
25.22 Product and service concept handling	35
25.23 Browser map behavior	35
25.24 Viewport loading	35
25.25 Binary snapshot in detail	35
25.26 Server startup after loading a snapshot	36
25.27 Progress bars, timings, and metrics	36
25.28 Memory reduction rules	36
26 Live verification summary	37
27 Commands	37
27.1 Build	37
27.2 Parse Europe	37
27.3 Load and serve	37

27.4 Open browser	37
28 Closing mental model	38

1 One-page mental picture

The program is a local map engine. It does not call Google, Nominatim, Overpass, or any internet geocoding service. It reads an OpenStreetMap PBF file, keeps the objects that the local geocoder and map viewer actually use, writes a binary snapshot, loads that snapshot, and serves a browser map.

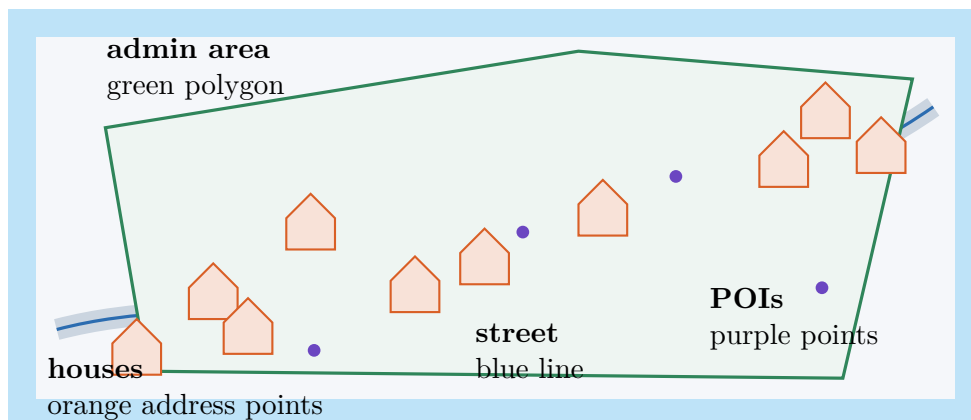


Figure 1: The reduced dataset shown as the program sees it: address points, street lines, admin polygons, and searchable POIs.

2 Input file: what an OSM PBF contains

2.1 PBF file

.osm.pbf is a compressed binary OpenStreetMap file. It is not an image. It stores billions of structured objects. The implementation uses **libosmium** to read this file.

The program reads three OSM primitive types:

Primitive	What it is	How this program can use it
Node	One coordinate with tags	house point, POI point, coordinate needed by a way
Way	Ordered list of node IDs plus tags	street line, building address, POI area, admin boundary piece
Relation	Group of nodes/ways/relations with member roles	administrative multipolygon, building relation address

2.2 Node drawing

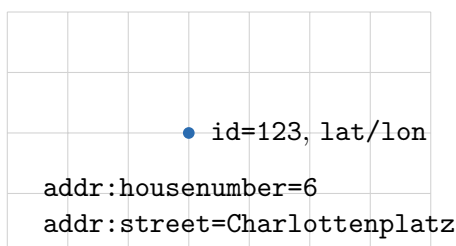


Figure 2: A node is one point. If it has address tags, the parser can store it as a house record.

2.3 Way drawing

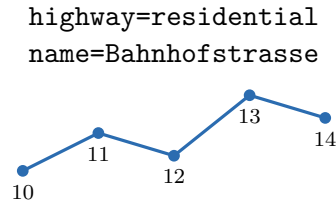


Figure 3: A way stores node IDs. The parser must recover the coordinates before it can draw a street line.

2.4 Relation drawing

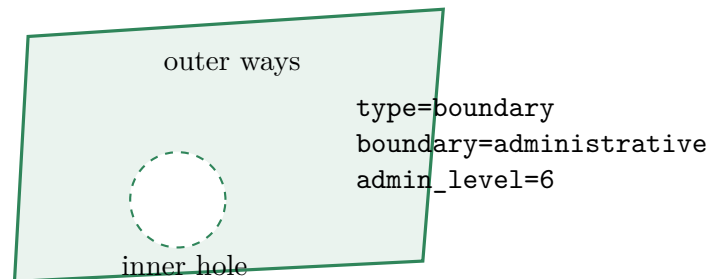


Figure 4: An administrative relation can have outer rings and inner rings. The implementation stores both.

2.5 Tags

Tags are key-value pairs. The parser does not understand every possible tag. It keeps tags that are needed for houses, streets, admin areas, POIs, display names, ranking, and metrics.

Tag group	Why it matters
<code>addr:housenumber</code> , <code>addr:street</code>	Makes an OSM object useful as an address/house record.
<code>addr:postcode</code> , <code>addr:city</code> , <code>addr:country</code>	Strengthens address lookup and display.
<code>building=*</code>	Lets addressed ways/relations become house records.
<code>highway=*</code>	Lets ways become street records, unless the highway type is skipped.
<code>name=*</code> , <code>official_name=*</code> , <code>ref=*</code>	Names and road references for display and search.
<code>name:de=*</code> , <code>name:en=*</code>	Localized names when available. The selected parser language is German by default.
<code>boundary=administrative</code> , <code>admin_level=*</code>	Creates admin areas used for hierarchy and ranking.
<code>amenity=*</code> , <code>shop=*</code> , <code>tourism=*</code> , <code>leisure=*</code> , <code>landuse=*</code>	Creates selected POIs.
<code>brand=*</code> , <code>operator=*</code>	Helps POI display and brand search.

3 Build and runtime shape

3.1 Executable

The executable is `osm_parser`. The build uses C++17, libosmium, protozero, zlib, bzip2, expat, cpp-httplib, and threads.

3.2 Main command modes

Mode	Meaning
<code>-parse input.osm.pbf</code>	Read the PBF and build the compact dataset.
<code>-load-binary file.bin</code>	Load an already written compact dataset.
<code>-save-binary file.bin</code>	Save the compact dataset, including the forward geocoder index.
<code>-server 8080</code>	Start the local HTTP server and browser UI.
<code>-low-memory</code>	Use extra sequential scans to reduce RAM pressure while parsing.
<code>-connect-streets</code>	Merge touching street segments with the same identity.
<code>-pbf-threads n</code>	Configure libosmium PBF decoding threads.
<code>-geojson out.geojson</code>	Export a bounded debugging GeoJSON sample.

4 Parser: what it keeps

4.1 House records

A **house** in this program means an address object, not only a detached house. The parser keeps an OSM object as a house when it has `addr:housenumber` or `addr:street`. For ways and relations, it also requires `building=*`.

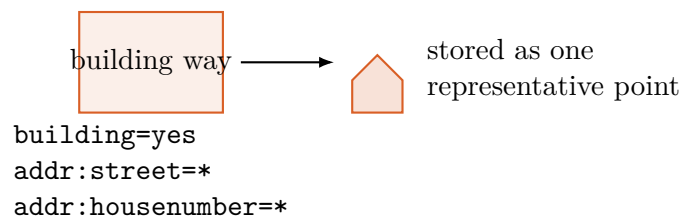


Figure 5: A building polygon with address tags is reduced to one address point.

Stored fields:

- OSM ID
- representative coordinate
- house number
- street name
- postcode
- city
- country
- admin area link range

- source type: node, way, or relation

The parser keeps houses even when the street or house number is missing. Those incomplete addresses are weaker for exact forward geocoding, but useful for reverse geocoding and for measuring OSM data completeness.

4.2 Street records

A street record comes from an OSM way with `highway=*`, unless it is blocked or area-like.

The parser keeps:

- OSM ID
- street display name
- highway type
- geometry offset and size
- bounding box
- admin area link range

Street names are chosen with this order:

1. localized `name:<language>` when present
2. `name`
3. localized `official_name:<language>` when present
4. `official_name`
5. `ref`, such as A 8, B 14, or L 558

4.3 Admin area records

Admin areas come from:

- administrative boundary ways
- administrative boundary relations

They require `boundary=administrative` and a valid `admin_level` from 2 to 12.

Stored fields:

- OSM ID
- name
- admin level
- geometry/ring offsets
- parent area link range
- bounding box
- source type

4.4 POI records

POI means point of interest. The parser stores selected real-world places, not every tagged object.

Stored POI groups:

- **amenity**: fast food, restaurant, cafe, bar, pub, fuel, hospital, school, university, pharmacy, bank, parking
- **shop**: any non-empty shop except **shop=no**
- **tourism**: hotel, museum, attraction
- **leisure**: park, garden, playground, sports centre, swimming pool
- **landuse**: recreation ground, village green, stored as park-like

The POI display name is chosen from localized name, normal name, localized brand, brand, localized operator, operator.

5 Parser: what it skips

5.1 Skipped street-like ways

The implementation skips street ways that do not help the intended road/address layer.

Skipped case	Reason
area=yes, area=1, area=true	A plaza or road area is not a centerline street.
highway=construction	Not a usable final road.
highway=proposed	Future road, not current navigable data.
path, footway, cycleway, bridleway, steps	Not part of the selected street/address layer.
platform, corridor	Indoor/transport/platform geometry, not normal street centerline.
Unnamed highway ways	Normally skipped unless they match the unnamed recovery rule.

5.2 Unnamed street recovery

Some useful connector roads have no name. The parser can recover selected unnamed ways when they are useful connectors:

- motorway/trunk/primary/secondary/tertiary link ways
- unclassified, residential, living street, road
- roundabouts

This keeps map connectivity better without accepting every footpath or cycle path.

5.3 Skipped POIs

The parser skips POIs outside the selected tag rules. For example, random tags such as **bench=yes**, **surface=asphalt**, or **lit=yes** are not stored as searchable POIs. They are valid OSM data, but they are outside the program's search model.

5.4 Skipped raw data

The parser does not keep:

- raw node table for all Europe nodes

- all OSM tags
- complete OSM metadata such as changeset/user/version/timestamp
- relation members after assembly
- geometry for buildings after one representative address point is created
- every non-address building footprint

Those choices are deliberate: the server needs address, street, admin, POI, ranking, and display data, not a full OSM database clone.

6 Low-memory PBF reading

6.1 Why multiple scans exist

An OSM way contains node IDs, not coordinates. The simple method would load every node coordinate into a giant map. Europe has billions of nodes, so that would push RAM and swap too hard.

The implementation uses compact needed-node recovery:

1. scan ways and relations to learn which node IDs are needed
2. sort and bucket those IDs
3. scan nodes and fill only those coordinates
4. build the final ways/records using the compact node store

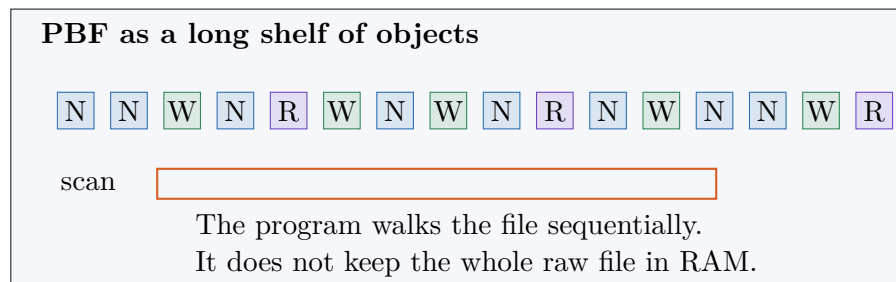


Figure 6: Sequential scans are cheaper than holding every raw node coordinate in memory.

6.2 NeededNodeStore

`NeededNodeStore` stores only node IDs required for selected ways and relations. It uses:

- sorted node IDs
- coordinate array
- compact found-bit storage
- bucket starts for faster lookup

This is why `-low-memory` is recommended for Europe on a laptop. It adds scans but avoids memory blowups.

7 Administrative boundary assembly

7.1 Relation way spans

Admin relations reference member ways. The implementation stores relation member geometry in flat arrays:

- `RelationWaySpan`: way ID, offset, size
- `relationWayCoords`: all coordinates in one vector

This avoids one small heap allocation per member way.

7.2 Ring building

Member ways are stitched into closed rings. A valid ring has at least 4 coordinates because the last coordinate closes the polygon.

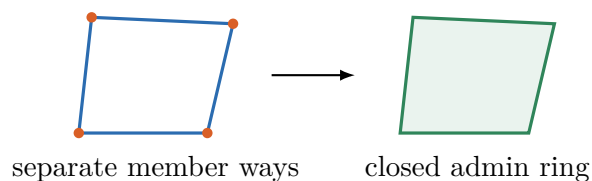


Figure 7: Boundary member ways are assembled into polygon rings before point-in-polygon checks.

7.3 Inner holes

Inner rings are assigned to the smallest outer ring that contains a probe point from the inner ring. At lookup time, a point is inside an admin area only if:

1. it is inside at least one outer ring
2. it is not inside any inner ring belonging to that admin area

8 Point-in-polygon and admin links

8.1 Point-in-polygon test

The code uses a ray-crossing style polygon test. Instead of checking every polygon edge for every point, it also builds an edge index by latitude. A point only tests edges near that latitude.

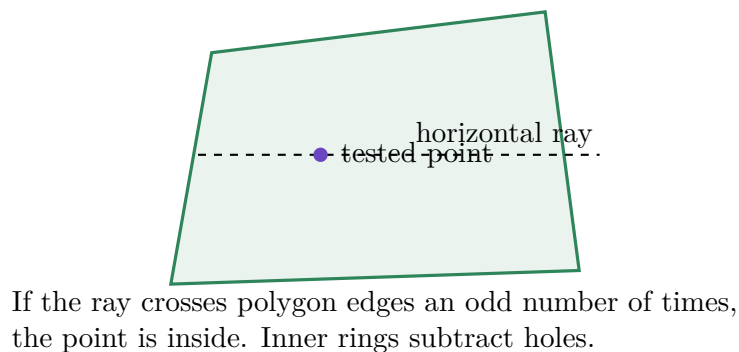


Figure 8: Admin containment depends on real geometry, not only bounding boxes.

8.2 Admin links for objects

The parser assigns:

- houses to containing admin areas
- street representative points to containing admin areas
- POIs to containing admin areas
- admin areas to parent admin areas

Bounding boxes and grid indexes prune candidates first. Polygon tests then confirm containment.

9 Street connection

9.1 Why streets are connected

OSM streets are often split into many ways. With `-connect-streets`, the program can merge touching street segments when they have the same name/highway identity.

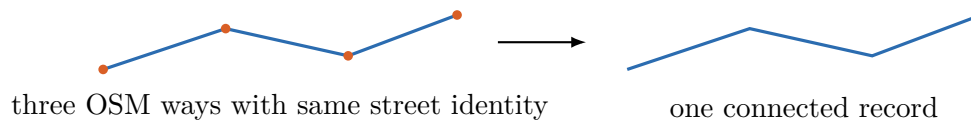


Figure 9: Street connection reduces fragmented line records when endpoints touch.

9.2 How connection works

The implementation builds endpoint records, groups segments with equal identity, and uses a disjoint-set structure. Segments in the same connected component are merged into a connected street geometry. After connection, admin attribution is recomputed and the forward index is cleared so it can be rebuilt from the final street list.

10 Compact dataset memory model

10.1 String table

Repeated text is stored once in `data.strings`. Records store `StringRef`, a 32-bit number pointing into that table.

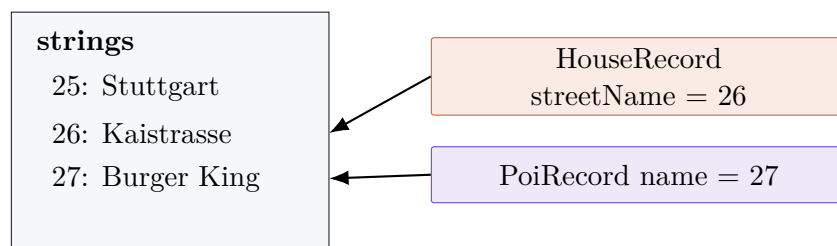


Figure 10: Records hold small references instead of full repeated strings.

10.2 Geometry arrays

Street and admin coordinates are stored in shared coordinate vectors. Records store offset and size.

- `streetGeometry`: all street line coordinates
- `adminGeometry`: all admin ring coordinates
- `geometryOffset`: where this feature starts
- `geometrySize`: how many coordinates it owns

10.3 Checked 32-bit offsets

Offsets and sizes are stored as `uint32_t`. The implementation uses checked casts when writing offsets, sizes, indexes, and link ranges. If a vector grows past the 32-bit range, the program throws an error instead of silently wrapping.

11 Binary snapshot

11.1 What is written

The binary snapshot stores:

- magic header
- selected name language
- string table
- houses
- streets
- admin areas
- POIs
- admin link arrays
- street geometry
- admin geometry
- admin rings
- statistics
- forward geocode index

11.2 Why the forward index is inside the snapshot

The forward index is expensive for Europe because it touches houses, streets, admin areas, and POIs. The final implementation writes it into the binary snapshot. Loading the binary no longer rebuilds the whole forward text index from scratch.

11.3 What still builds at server startup

The server still builds live spatial indexes:

- house grid
- street grid
- admin grid
- POI grid

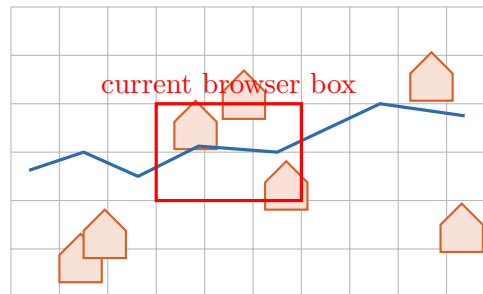
These are server runtime structures used for viewport loading and reverse lookup.

12 Server startup

12.1 Server constructor

When the server starts, it receives a const reference to the loaded dataset. It does not copy the whole dataset. It builds runtime indexes that point into existing record vectors.

12.2 Spatial grid drawing



Only cells touching the viewport are inspected for map loading.

Figure 11: The grid keeps browser requests from scanning the whole Europe dataset.

13 HTTP API

13.1 Stats endpoint

`/api/stats` returns counts, timings, memory estimates, and server index build metrics. The frontend uses it for the left statistics panel.

13.2 Viewport endpoints

- `/api/houses`: returns address points inside the map window
- `/api/streets`: returns street geometry inside the map window
- `/api/admin`: returns admin areas intersecting the map window
- `/api/admin-area`: returns one admin area geometry, optionally simplified

The frontend asks only for the current map box. This is why the browser can display Europe without downloading millions of objects at once.

13.3 Search endpoints

- `/api/geocode`: deterministic text search
- `/api/natural-geocode`: optional local LLM intent search
- `/api/reverse`: coordinate to nearest address/street/admin result

14 Frontend map

14.1 Layer display

The frontend is a single HTML page using Leaflet. It displays:

- orange house markers
- blue street polylines
- green admin polygons
- purple geocoder result markers
- reverse lookup popups

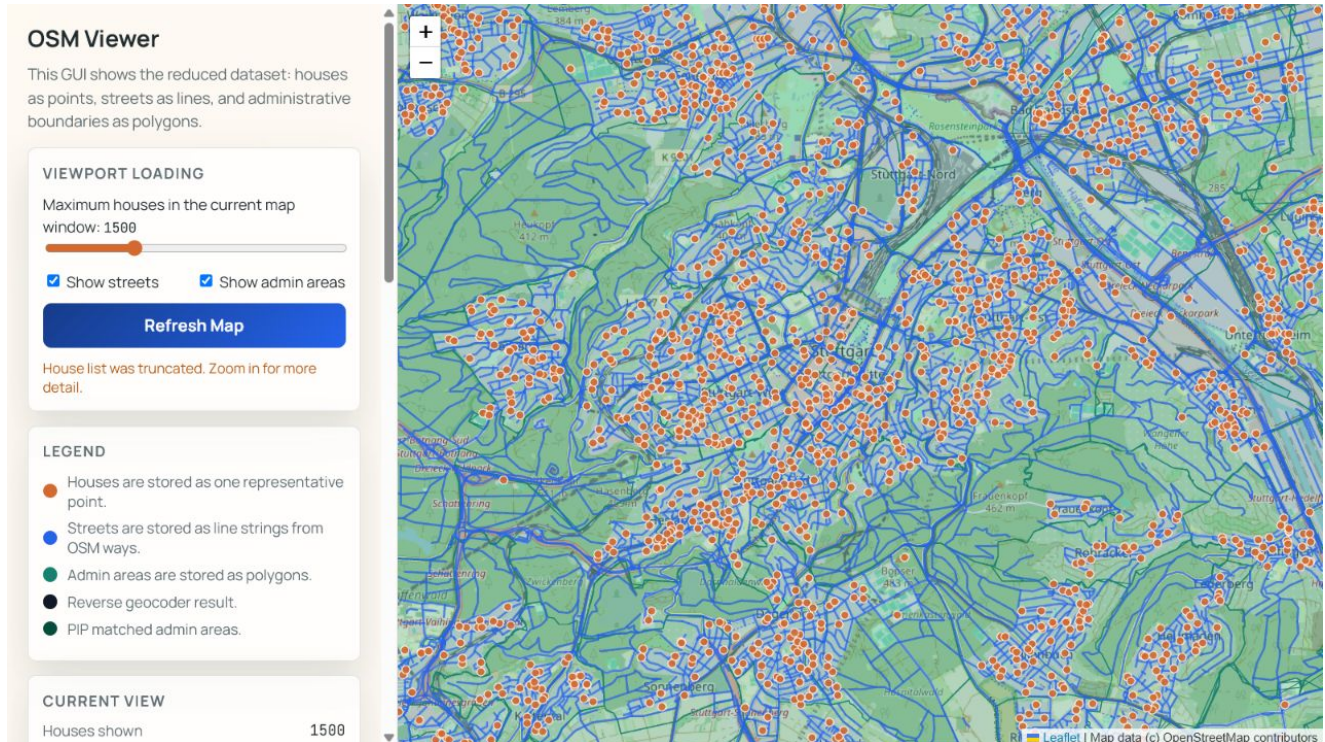


Figure 12: The running browser UI with houses, streets, admin polygons, controls, and statistics.

14.2 Viewport loading control

The slider limits how many house points are loaded in the current map window. If the map is zoomed out too far, the server can return a truncated set and the UI tells the user to zoom in for detail.

14.3 Result highlighting

Search results are drawn with highlighted markers. Admin result polygons can be fetched separately to avoid sending heavy geometry for every search result.

15 Forward geocode index

15.1 Inverted index

The forward geocoder uses an inverted index. A token points to records that contain that token.

15.2 Context index

The context index stores broad search context:

- house street, number, postcode, city, country, admin names

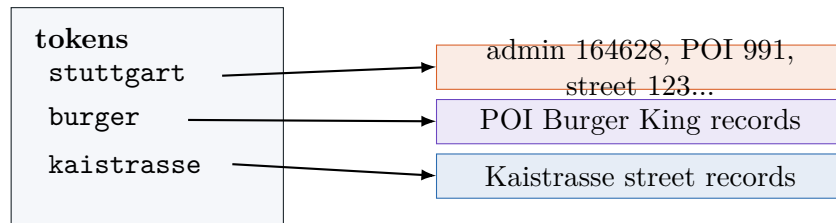


Figure 13: The geocoder starts from text tokens and jumps straight to candidate records.

- street name and admin names
- POI name, brand, category, tag value, admin names
- admin area name and parent names

15.3 Primary index

The primary index stores stronger object-owned fields:

- house street, number, postcode
- street name
- POI name, brand, category, tag value
- admin area name

Single-token queries without numbers or postcodes use the primary index first. Multi-token and address-like queries use the context index.

15.4 Substring suffix array

For substring search, the program stores suffixes of vocabulary tokens with length at least 3. A query fragment can binary-search this suffix array and expand to real tokens that contain the fragment.

15.5 Fuzzy search

Fuzzy search uses bounded edit distance:

- token length under 4: no fuzzy expansion
- token length 4 or 5: max edit distance 1
- longer token: max edit distance 2
- vocabulary scan is capped at 250,000 tokens

Fuzzy search only returns tokens already present in the local index.

16 Forward query matching

16.1 Normalization

Text is normalized before indexing and querying:

- ASCII letters become lowercase
- punctuation becomes spaces
- German umlauts become **ae**, **oe**, **ue**

- **ss** is used for **ß**
- duplicate tokens are removed
- **str**, **strasse**, and related street spelling normalize toward **strasse**

16.2 Candidate intersection

The geocoder finds a posting list for each query token. It sorts posting lists by size and uses the smallest list as the anchor. A candidate must appear in every token posting list to pass the first stage.

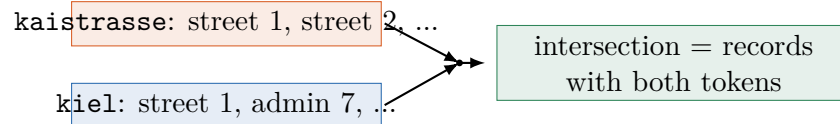


Figure 14: Candidates must match all query tokens before scoring.

17 Forward ranking formulas

17.1 House ranking

House candidates are considered only when the query has a number. The number must match. If a postcode is present, the postcode must match.

House score:

$$\begin{aligned} \text{score} = & 120 + 15 \cdot \text{matchedTokens} + 80 \cdot \text{numberMatch} \\ & + 45 \cdot \text{postcodePresent} + 25 \cdot \text{streetMatches} \\ & + 12 \cdot (\text{cityMatches} + \text{adminMatches}) \end{aligned}$$

This makes exact numbered addresses stronger than loose text matches.

17.2 Street ranking

Street score:

$$\begin{aligned} \text{score} = & \text{base} + 15 \cdot \text{matchedTokens} \\ & + 25 \cdot \text{streetMatches} + 10 \cdot \text{adminMatches} \end{aligned}$$

Here **base** is 35 for number queries and 95 otherwise.

Additional terms:

- exact street name: +50
- street phrase appears in full query: +220 plus +160 per street token
- street name looks like a place name when query wants something else: -420
- result mostly matches only a place token: -420

17.3 Admin ranking

Admin score:

$$\begin{aligned} \text{score} = & \text{base} + 15 \cdot \text{matchedTokens} \\ & + 25 \cdot \text{nameMatches} + 8 \cdot \text{parentMatches} \end{aligned}$$

Here **base** is 135 for one-token place queries, 75 for other non-number queries, and 10 for number queries.

Additional terms:

- higher-level admin preference: $\max(0, 14 - \text{adminLevel}) * 3$
- exact area name: +60
- known place exact match: +2600
- known place full token match: +900

This is why **stuttgart** ranks the actual Stuttgart admin area above a random POI named Stuttgart in another city.

17.4 POI ranking

POI score:

$$\text{score} = \text{base} + 15 \cdot \text{matchedTokens} + 30 \cdot \text{poiMatches} + 12 \cdot \text{adminMatches}$$

Here **base** is 20 for number queries and 105 otherwise.

Additional terms:

- exact POI display name: +55
- POI text relevance: name/brand/operator/category strength
- POI query type score: favors the category implied by the query
- map-center focus score: up to +180 based on distance
- inside current viewport: +160
- known place query and POI admin matches that place: +420
- known place query and POI admin does not match: -1900

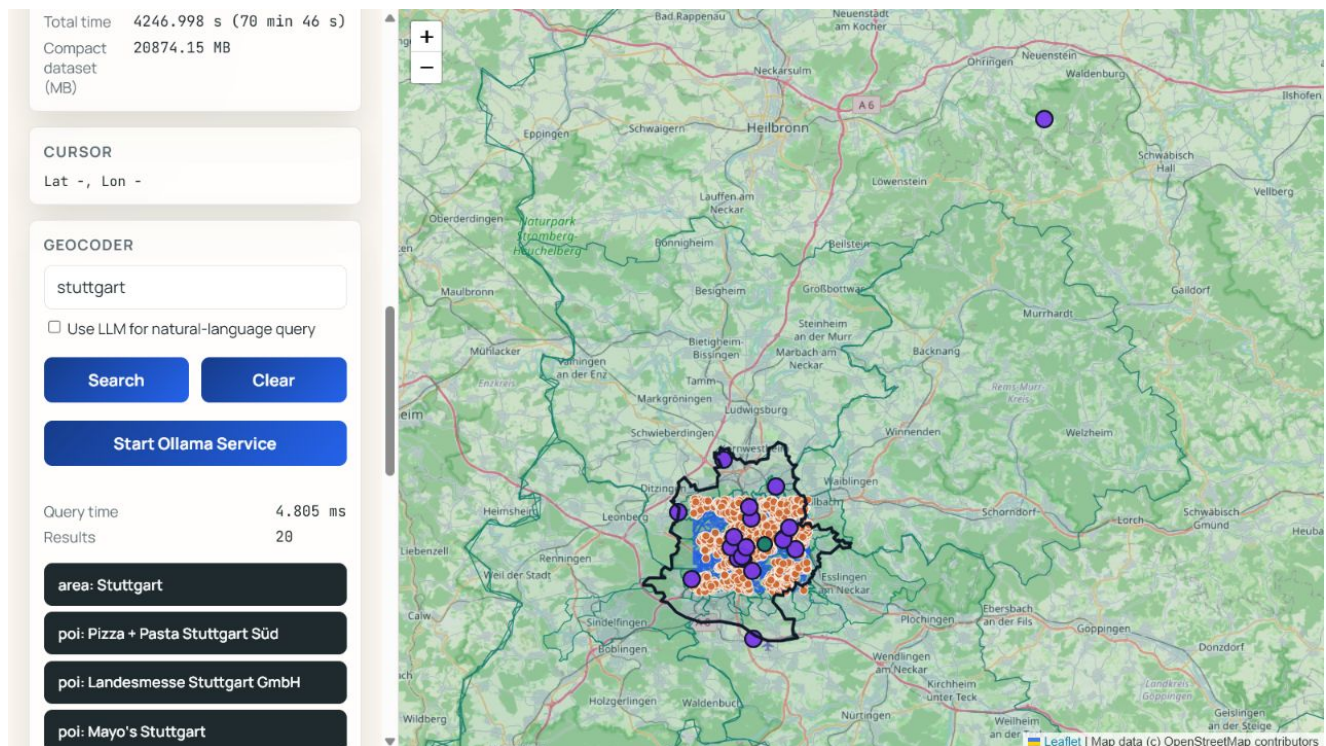


Figure 15: The browser result for **stuttgart**: admin area first, then POIs related to Stuttgart.

17.5 Additional candidate helpers

The geocoder can add candidates after the strict intersection:

- street-place helper for street plus place queries
- brand-place helper for brand in city queries
- English/German place-token handling for names like `munich/muenchen`

These helpers still return only records from the local PBF-derived dataset.

18 Reverse geocoder

18.1 Input and output

Reverse geocoding receives a coordinate:

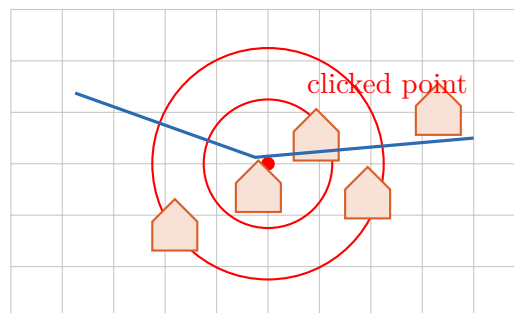
```
/api/reverse?lat=48.7758&lon=9.1829
```

It returns:

- whether something was found
- closest house when available
- closest street when available
- matched admin areas
- distance values
- query time

18.2 Distance search

The reverse geocoder checks nearby spatial grid cells. It does not scan all houses and all streets.



Search expands through nearby grid cells until enough candidates are checked.

Figure 16: Reverse lookup expands around the clicked point instead of scanning the full dataset.

19 Natural-language and Ollama mode

19.1 What Ollama is allowed to do

Ollama is used only for intent parsing. It does not return final places. The backend still searches local OSM indexes and validates the result.

The LLM can produce these intent types:

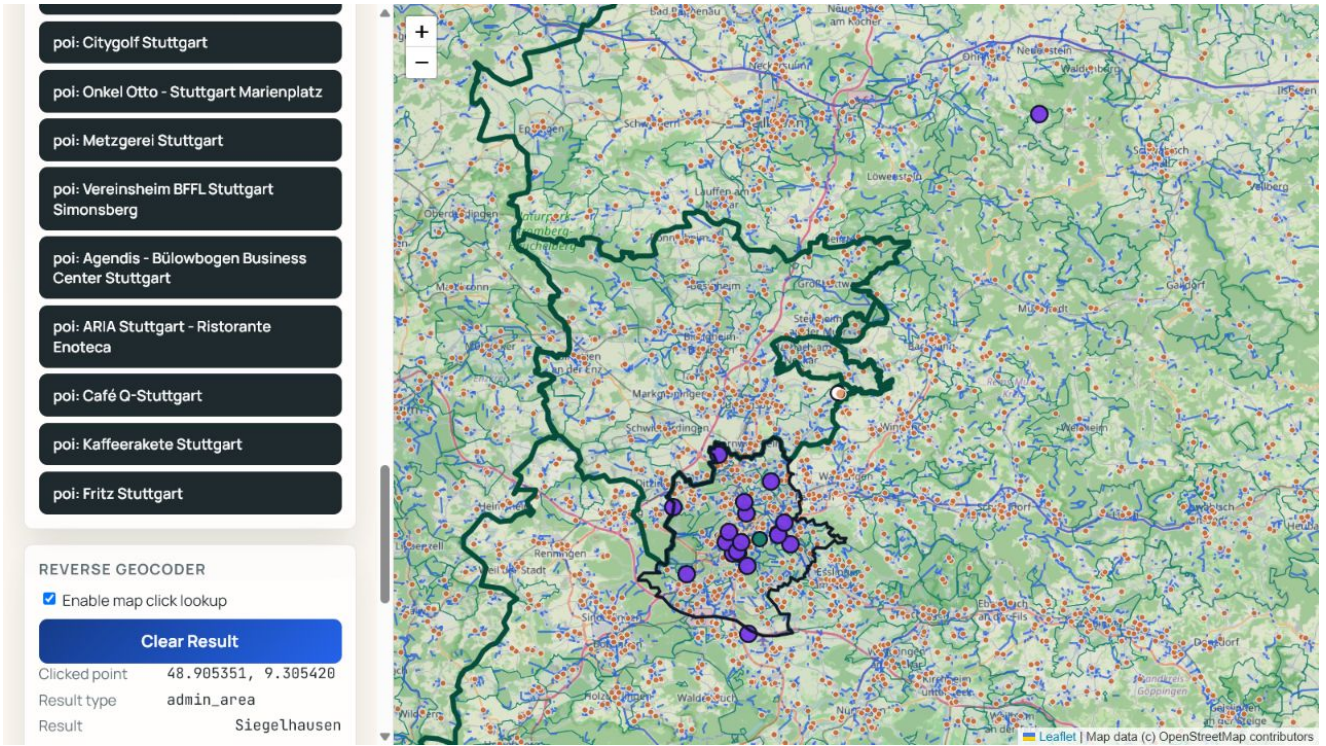


Figure 17: A reverse lookup in the running browser UI.

- named POI or brand in a place
- nearest category to address
- nearest product/service concept to address or viewport
- plain address-shaped query

19.2 Two verification passes

The program asks Ollama to draft strict JSON, then asks it to verify/correct that JSON twice. The final JSON is parsed by the backend. Invalid or unsupported JSON does not directly become a result.

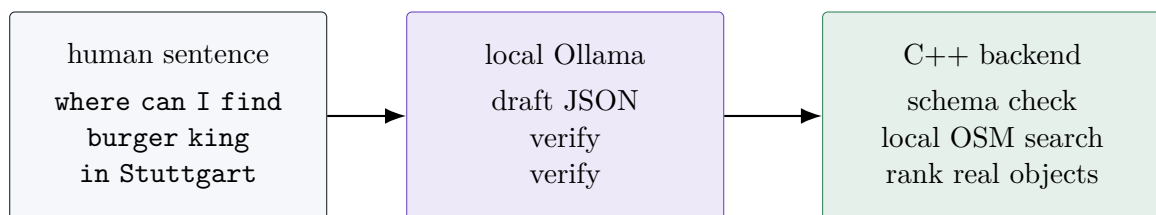


Figure 18: The LLM translates intent; the server decides using local OSM data.

19.3 Natural address guard

Short queries like *Kaistrasse 5 Kiel* are address-shaped. If the LLM classifies them as a POI search, the backend overrides that path and treats them as address lookup. If the exact house number is missing, it retries without the house number and can return the street.

19.4 Product and service concept graph

Product/service queries map user words to OSM tag families. Example:

- medicine or cough syrup can map toward pharmacy/chemist
- bread can map toward bakery/supermarket
- bike repair can map toward bicycle shop/service
- printing documents can map toward copyshop/print service

This does not prove inventory. It returns likely real OSM places based on category tags.

19.5 Local knowledge graph in depth

In this project, the knowledge graph is not a remote database and not a learned model. It is a local set of typed concepts and allowed connections used by the backend to turn messy product/service wording into OSM-searchable tags. Ollama may suggest an intent, but the backend checks the intent against this graph before it searches.

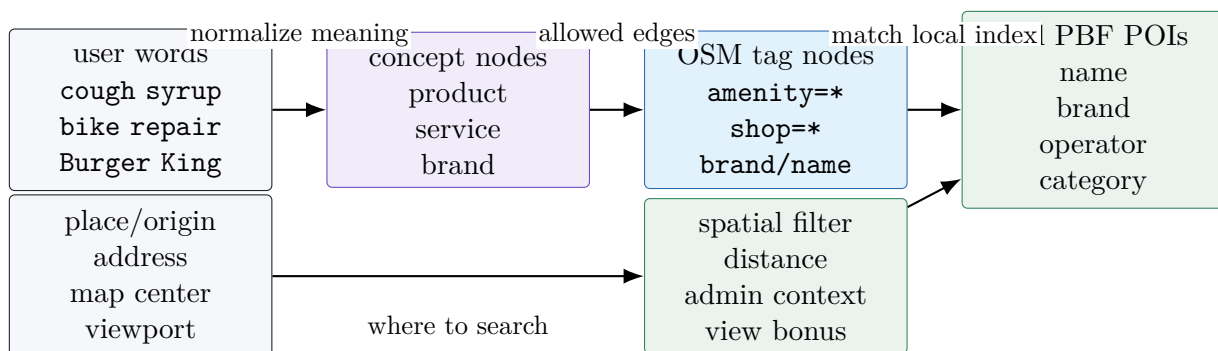


Figure 19: The local knowledge graph connects human words to allowed OSM tag families and then to real POIs from the loaded PBF.

Graph node types

The graph uses several node types:

- **Product concept:** a thing the user wants to buy or find, such as medicine, bread, groceries, electronics, or fuel.
- **Service concept:** an action the user wants, such as bike repair, document printing, eating, charging, parking, or banking.
- **Brand concept:** a name that may appear in **name**, **brand**, or **operator**, such as a restaurant chain, supermarket, car brand, parcel service, or hotel brand.
- **Category concept:** a local category used by the program, such as restaurant, fast food, pharmacy, shop, hotel, fuel, park, or museum.
- **Place/origin concept:** the location context, which can come from a typed place name, an address, the current map center, or the current viewport.
- **OSM tag concept:** a searchable OSM field such as `shop=*`, `amenity=*`, `tourism=*`, `leisure=*`, `landuse=*`, `name`, `brand`, or `operator`.

Graph edge meanings

Edges are not proof that a product exists at one exact shop. They are allowed search directions. For example:

- **product likely sold at category:** medicine can search pharmacies/chemists; bread can search bakeries or supermarkets.
- **service provided by category:** bike repair can search bicycle shops or repair-related POIs.
- **brand matches text fields:** a brand query searches **name**, **brand**, and **operator**.
- **category maps to OSM tags:** fast food maps toward **amenity=fast_food**; hotel maps toward **tourism=hotel**; pharmacy maps toward **amenity=pharmacy**.
- **origin limits or biases results:** an address or map center gives the nearest-category search a starting point; a named place gives admin context.

How Ollama output is checked against the graph

Ollama receives the sentence and is asked for strict JSON. The backend does not accept free text. The JSON must name an allowed intent, such as named POI/brand search, nearest category to address, nearest concept to address or viewpoint, or address-shaped lookup.

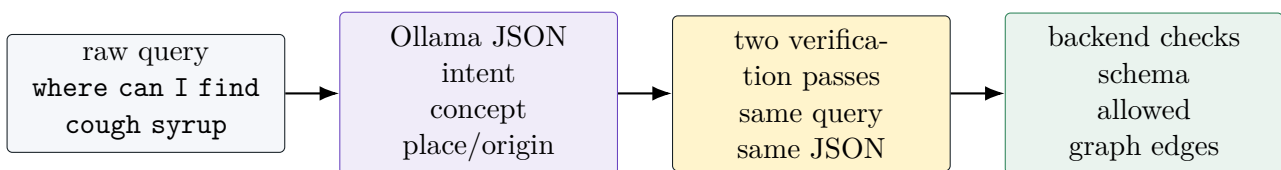


Figure 20: Ollama proposes structured intent; the backend verifies the shape and rejects unsupported graph paths.

The backend checks:

- whether the intent name is supported
- whether required fields are present
- whether the concept/category can be mapped to local OSM tags
- whether a place or address origin can be resolved by the local geocoder
- whether the final candidates exist in the loaded PBF

If any of these checks fails, the program does not invent a result. It can return no result, a weaker category result, or an address/street-level result when that is the correct local-data answer.

Candidate selection from graph-derived tags

After validation, the graph produces search handles. These handles are not final answers. They are only candidate selectors.

1. A product/service concept chooses allowed categories and tag families.
2. A brand concept chooses text fields: **name**, **brand**, and **operator**.
3. A place or address chooses an origin and admin context.
4. The server searches POI indexes and spatial grids for candidates.
5. Each candidate is still a real OSM object from the binary snapshot.

How graph relevance enters ranking

The ranker combines graph relevance with the normal geocoder signals:

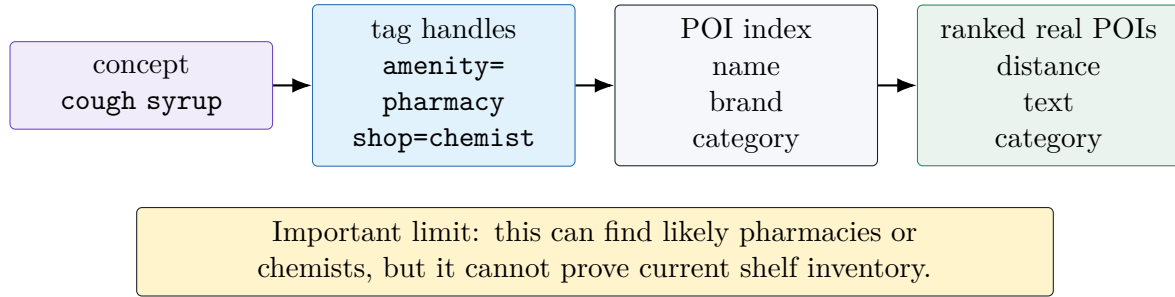


Figure 21: Graph-derived tags select candidate POIs; ranking decides which real local objects appear first.

- **text relevance:** the candidate name, brand, operator, category, or tag value matches the query words.
- **graph relevance:** the candidate category/tag is one of the allowed graph targets for the concept.
- **admin relevance:** the candidate lies inside or near the named place context.
- **distance relevance:** nearest-category searches prefer candidates closer to the resolved origin.
- **viewport relevance:** ambiguous searches can prefer candidates near the current map view.
- **exact brand relevance:** a direct brand/name match beats a broad category match.

This is why where can I find burger king in Stuttgart should favor Burger King POIs in or near Stuttgart, while where can I find burger can search broader restaurant or fast-food candidates.

What the graph deliberately does not do

The graph does not call the internet, does not query store catalogues, does not check opening hours in real time, and does not create synthetic POIs. It only changes how the local OSM data is searched and ranked. The final visible result must still be a real object stored in the loaded binary snapshot.

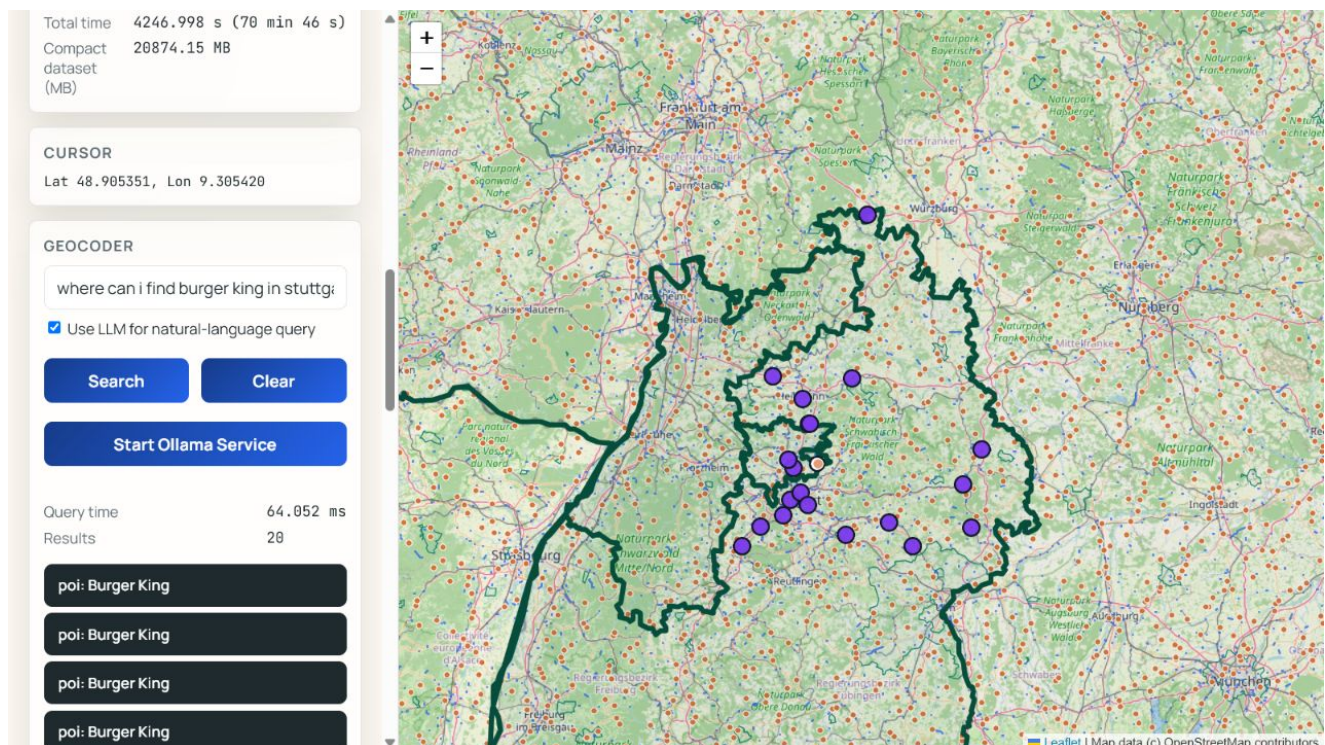


Figure 22: LLM mode in the browser: the natural sentence becomes local OSM POI results.

20 Frame of reference

20.1 Map center

The frontend sends map center coordinates with geocoder requests. This lets ranking favor nearby POIs when the query is ambiguous.

20.2 When place names override the current view

If the query clearly names a known place, the program should not trap the result inside the current view. For example, searching `stuttgart` while looking at Barcelona should still return Stuttgart.

20.3 When the current view helps

If the user searches `Burger King` while looking at Stuttgart, a nearby Stuttgart Burger King should rank higher than a distant one.

21 Browser controls

21.1 Geocoder text field

The text field sends to:

- `/api/geocode` when LLM checkbox is off
- `/api/natural-geocode` when LLM checkbox is on

21.2 Start Ollama Service button

The frontend includes a button to start the local Ollama service. The server-side handler can clear the occupied Ollama port and start the service. This is useful when the browser is open but Ollama was not running.

21.3 Reverse geocoder checkbox

When enabled, map clicks call `/api/reverse`. When disabled, clicking the map does not run reverse lookup.

22 Performance controls

22.1 Progress bars

Parser and index phases print progress bars with:

- phase number
- phase name
- percentage
- completed bytes or records
- speed
- elapsed time

- ETA when enough progress exists

22.2 Why ETA can be unknown

At the very beginning there is not enough completed work to calculate a meaningful speed. ETA becomes meaningful only after completed units and rate are both non-zero.

22.3 Why speed can drop during parsing

The displayed speed is not pure disk speed. A phase may start with fast sequential decompression and later slow down when it performs more filtering, node lookup, geometry extraction, string interning, or memory writes.

23 Memory controls

23.1 Released string interner index

During parsing, `StringInterner` needs a hash map to avoid duplicate strings. After strings are released into `data.strings`, the hash map index is cleared before admin attribution. This lowers peak memory during the expensive geometry lookup phase.

23.2 Flat relation geometry

Relation member geometry uses flat spans plus a coordinate vector instead of a map from way ID to a vector. This reduces small allocations and duplicate vector overhead.

23.3 Moved posting lists

Forward index finalization moves posting vectors from temporary maps into final posting lists. It does not copy every posting vector again.

23.4 Packed geocode refs

`GeocodeRef` is packed to 5 bytes:

- 1 byte type
- 4 bytes record index

This matters because Europe has many posting references.

24 Exact behavior examples

24.1 Kaistrasse 5 Kiel

The direct deterministic query `Kaistrasse 5 Kiel` can return zero if the exact house number is missing from OSM. With LLM mode enabled, the backend recognizes the short query as address-shaped and retries `Kaistrasse Kiel`. The result is the Kiel street when the exact house point is absent.

24.2 Stuttgart from Barcelona

The query `stuttgart` is a known place query. Admin area ranking gets a strong exact-place boost. POIs outside the matching admin chain are penalized. The result is the Stuttgart admin area, not a random Barcelona POI named Stuttgart.

24.3 Stuttgart Burger King

The query contains a place and a brand. The geocoder uses POI name/brand/category tokens plus admin context. Burger King records inside or near Stuttgart rank strongly.

25 Complete implementation walkthrough

This section is the slow, detailed version of the program. It explains each feature as a mechanical process: what the feature means, what input it reads, what data is accepted, what data is rejected, what arrays or indexes are changed, and what the browser or API can do with the result.

25.1 PBF primitives in the program

OpenStreetMap stores map data as three primitive types.

- A **node** is one coordinate pair. The parser uses nodes for address points, POI points, and geometry points used by ways.
- A **way** is an ordered list of node IDs. The parser uses ways for roads, building outlines, POI areas, and simple admin boundary lines.
- A **relation** is a list of other nodes, ways, or relations with roles. The parser uses relations for administrative multipolygons and addressed building relations.

The important detail is that a way usually stores only node IDs, not the coordinates of those nodes. That is why the parser cannot read Europe in one simple pass and immediately know every road shape. It first learns which node IDs it needs, then scans the nodes to recover their coordinates.



Figure 23: A road or boundary way points to nodes. The parser must recover node coordinates before it can draw geometry.

25.2 Tag reading and name selection

Every OSM object has a small dictionary of tags. The parser does not copy all tags into the final dataset. It reads tags only long enough to decide whether an object matters and which compact fields should be stored.

Tag group	How the program uses it
<code>addr:*</code>	Builds house/address records. <code>addr:housenumber</code> , <code>addr:street</code> , <code>addr:postcode</code> , <code>addr:city</code> , and <code>addr:country</code> become string references in <code>HouseRecord</code> .

<code>name, name:de, name:en</code>	Chooses display/search names. The selected language is tried first, then plain <code>name</code> , then selected <code>official_name</code> , plain <code>official_name</code> , and finally road <code>ref</code> when needed.
<code>highway</code>	Decides whether a way is a street and whether it is a real line road or an area/path that should be skipped.
<code>boundary=administrative, admin_level</code>	Decides whether a way or relation is an administrative area. Levels outside the accepted integer range are ignored.
<code>amenity, shop, tourism, leisure, landuse</code>	Decides whether an object becomes a searchable POI and which local category it receives.
<code>brand, operator</code>	Helps POI and natural-language searches. A query such as a chain name can match these fields even when the visible <code>name</code> differs.

The code uses normalized search text only for indexing and matching. The original readable string remains in the string table so the browser can show the real name with accents and punctuation.

25.3 Address extraction from nodes

A node becomes a house record when it carries address tags. The parser checks the tags first, then stores the coordinate directly because a node already has latitude and longitude.

1. Read a node from libosmium.
2. Check whether `addr:housenumber` or `addr:street` exists.
3. If neither field exists, the node is not an address and is skipped for house extraction.
4. If at least one address field exists, copy the coordinate into a `HouseRecord`.
5. Store address strings through the `StringIntern`; the record keeps integer references, not full string copies.
6. Mark the source as `node` so the summary can later say how many houses came from nodes.

This means a partial address is still stored. For example, a node with a street but no house number is useful for map display and reverse geocoding, even if it cannot satisfy an exact house-number query.

25.4 Address extraction from building ways

A building way is a polygon or outline made from node IDs. If the building has address tags, the parser stores one representative coordinate instead of keeping the whole building shape as a house. This keeps the final dataset small and makes point search fast.

1. Read a way and inspect its tags.
2. Require address tags and a building-like tag for addressed building extraction.
3. Add all node IDs from this way to the needed-node store.
4. During the node-coordinate recovery scan, fill the coordinates for those node IDs.
5. Compute a representative point from the available coordinates.
6. Store one `HouseRecord` with source `way`.

The representative point is enough for this program because the browser displays address dots and the reverse geocoder needs nearest address points. The full building polygon is not needed for the implemented geocoder behavior.

25.5 Address extraction from building relations

Some buildings are relations instead of plain ways. The parser handles addressed building relations separately because their members may be several ways.

1. The relation scan records building relations with address tags.
2. It remembers which member ways are needed for those relations.
3. When those member ways are later seen, their node IDs and coordinates are collected.
4. After way geometry exists, the parser computes one representative house coordinate for the relation.
5. The relation becomes a **HouseRecord** with source **relation**.

This avoids losing addresses that are modeled as multipolygon buildings.

25.6 Street extraction and street skip rules

A street is stored from a way when the way has an accepted **highway** value. The parser does not store every line-like object because Europe contains many paths, platforms, corridors, and proposed roads that would increase memory while making address search noisier.

1. Read a way and check **highway**.
2. Reject it if there is no **highway** tag.
3. Reject it if **area=yes**, **area=1**, or **area=true**; those are surfaces, not road centerlines.
4. Reject construction, proposed roads, paths, footways, cycleways, bridleways, steps, pedestrian areas, platforms, and corridors for the main street set.
5. Choose the street display name using the language/name rule.
6. If the street has a name or usable reference, store it as a named street.
7. If it has no name, store it only when unnamed connector recovery is enabled and the highway type is useful for connecting named road pieces.

The output is a **StreetRecord**. The record stores a name reference, a bounding box, a geometry offset, a geometry size, and a source object ID. The actual coordinates live in the shared **streetGeometry** array.

25.7 Unnamed street recovery

Many roads are split into several OSM ways. Sometimes a small connector piece has no name, even though it connects two named pieces of the same street network. If the program ignored all unnamed connectors, reverse geocoding and map display would show small gaps.

The parser keeps only selected unnamed connector types: link roads, residential connectors, living streets, unclassified roads, ordinary roads, and roundabout-like connector ways. It still skips footpaths, cycle paths, platforms, and proposed roads.

25.8 Street connection in full detail

Street connection is the process that merges road fragments that touch each other. It is used after street extraction, when all street records and their geometries already exist.

The process is:

1. For every street record, read the first and last coordinate from **streetGeometry**.

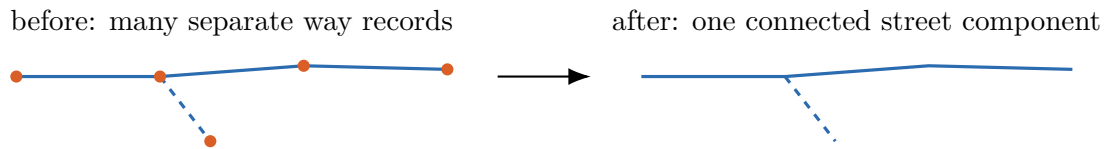


Figure 24: Street connection groups fragments when their endpoints touch. The geometry remains real OSM geometry; the program only groups matching pieces.

2. Convert each endpoint coordinate into a hashable key. The key uses the stored integer coordinate values, so endpoints that are identical in OSM become identical keys.
3. Build a map from endpoint key to the list of street record indexes that touch that endpoint.
4. Create a disjoint-set structure. At the start, every street record is its own component.
5. For each endpoint key, union all street records that share that endpoint.
6. Keep the union only when it is allowed by the connection rule. Named street pieces should not be merged blindly with unrelated names.
7. After all endpoint groups are processed, assign connected-component metadata.
8. The browser can still draw the original segments. The server can use the connected group to reason about streets that are split into many OSM ways.

The reason this is done after extraction is simple: the parser needs all streets first. A road segment at the beginning of the file might connect to a segment that appears much later in the PBF.

25.9 POI extraction step by step

A POI is a searchable place such as a restaurant, shop, fuel station, hotel, park, pharmacy, or museum. The program stores only POIs that have a meaningful local category and a coordinate.

1. Read node, way, or relation tags.
2. Look for accepted POI tags: selected **amenity**, most **shop** values, selected **tourism**, selected **leisure**, and selected **landuse**.
3. Convert the OSM tag into a compact category such as restaurant, shop, hotel, park, pharmacy, or fuel.
4. Choose a name. If there is no name, brand, operator, or useful category text, the object is less useful for search and may be skipped.
5. Store one coordinate: direct node coordinate for node POIs, representative coordinate for way/relation POIs.
6. Store name, brand, operator, category, tag key, and tag value through the string table.

The program does not store shop inventories. A drugstore POI can support a query for a likely drugstore product, but the data cannot prove that the exact product is on the shelf today.

25.10 Administrative boundary extraction

Administrative areas are countries, states, districts, cities, municipalities, and smaller named administrative regions. They are needed because addresses and streets are much clearer when the program knows which city or region contains them.

1. During the relation scan, keep relations with **boundary=administrative**.
2. Parse **admin_level**; reject invalid or out-of-range levels.

3. Record relation members with their roles, especially **outer** and **inner**.
4. When member ways are seen, store their coordinates in flat relation-way storage.
5. Assemble member way pieces into closed rings.
6. Store outer rings as the visible containing area.
7. Store inner rings as holes that subtract from the containing area.

Each final admin area has a bounding box, name, admin level, source ID, and ring metadata. The ring metadata is important because a multipolygon can have more than one outer ring and can contain holes.

25.11 Inner holes in admin areas

An inner ring means: this part lies inside the outer outline, but is not part of the admin area. The point-in-polygon logic therefore uses two tests.

1. A point must be inside at least one outer ring.
2. The same point must not be inside any inner ring assigned to that outer ring.

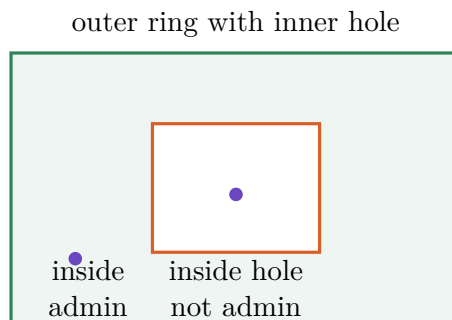


Figure 25: The outer polygon alone is not enough. The hole must be stored and subtracted.

25.12 Admin containment and object links

After admin geometry is assembled, the parser links houses, streets, POIs, and admin areas to the admin areas that contain them.

The process is:

1. Use the object's coordinate or representative coordinate.
2. Look up candidate admin areas through bounding boxes and grid pruning.
3. Run point-in-polygon only for candidates whose bounding box can contain the point.
4. Accept the admin area only when outer/inner ring logic says the point is truly inside.
5. Write compact links into arrays such as house-admin links and street-admin links.
6. Use those links later for display text, reverse geocoding, and ranking.

The program keeps links as arrays instead of storing a vector inside every object. That matters for Europe because millions of objects may have several admin ancestors.

25.13 Reverse geocoder step by step

Reverse geocoding starts with a clicked map point and answers: what is nearest here?

1. The browser sends latitude and longitude to `/api/reverse`.
2. The server checks nearby grid cells for house points, POI points, and street geometries.
3. For houses and POIs, distance is point-to-point.
4. For streets, distance is point-to-line-segment along the stored road geometry.
5. The nearest candidate of each type is kept.
6. Admin containment is looked up for the clicked point.
7. The JSON response returns the clicked coordinate, nearest objects, distances, and admin context.

The grid is what keeps this fast. The server does not scan every European street for every click.

25.14 Forward geocode index construction

The forward geocoder turns names and address fields into posting lists. A posting list is a list of object references that contain one normalized token.

1. Read every house, street, admin area, and POI.
2. Build normalized tokens from searchable fields.
3. Add object references into a temporary context map and a temporary primary map.
4. Move each map into sorted `GeocodePostingList` arrays.
5. Merge posting-list tokens into a sorted vocabulary.
6. Build a suffix array over vocabulary tokens for substring search.
7. Store the complete forward index inside the binary snapshot.

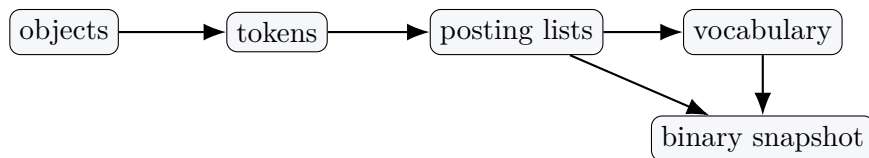


Figure 26: The forward index is built once during parsing and written into the binary snapshot.

25.15 Forward search query path

When the user searches without LLM mode, the program follows this path.

1. Normalize the query: lowercase, convert German umlauts into searchable forms, remove accents where possible, and split punctuation into spaces.
2. Split tokens into text tokens, house-number tokens, and postcode tokens.
3. Build token alternatives for English/German place spellings where the implementation has deterministic rules.
4. Find posting lists for the query tokens.
5. If a normal token lookup fails, use substring expansion for short contained words when allowed by the suffix array.
6. If substring still cannot recover a useful token, use bounded fuzzy matching for small spelling mistakes.
7. Pick the smallest posting list as the anchor.
8. Intersect candidates against the other required token posting lists.

9. Score each surviving candidate by object type and query shape.
10. Sort by score, then return the top results as JSON.

This is why query time stays low: the expensive work is the index build, not a full object scan on every search.

25.16 House ranking details

A house result is strong only when the query looks like an address. The ranking intentionally requires the house number to match if the user typed a number.

- Matching house number gives the largest address-specific boost.
- Matching postcode gives another strong boost when a postcode is present.
- Street-name token matches increase the score.
- City/admin token matches make the address more precise.
- If the user typed a number and the candidate has a different number, the candidate is rejected rather than weakly ranked.

This prevents a query like `Kaistrasse 5 Kiel` from returning a random nearby address with a different number as if it were exact.

25.17 Street ranking details

A street result is useful when the query asks for a street or when a specific house number is missing from OSM.

- Exact street-name phrase matches receive a strong boost.
- Street token matches count more than unrelated admin token matches.
- If the query looks like a known place name, street results that only contain that place token are penalized.
- If a query contains a number, street results can still appear as a fallback only when the exact house point is not available.

The `Kaistrasse 5 Kiel` behavior is an example: exact deterministic house search has no result because the house number is absent, but LLM address handling can still return the real street as a clearly street-level result.

25.18 Admin ranking details

Admin areas are ranked strongly for place-name queries.

- Exact admin-name match receives a large boost.
- Known-place queries receive an extra admin boost.
- Lower admin levels that represent larger or more recognizable areas are considered, but name match is still the main signal.
- POIs that happen to contain the same word are penalized when they are outside the matching admin context.

This is the reason `stuttgart` should rank the Stuttgart admin area above a Barcelona POI named Stuttgart.

25.19 POI ranking details

POI ranking combines text relevance, category relevance, brand/operator relevance, admin context, and map focus.

- Exact POI name matches are strong.
- Brand and operator matches help chain queries such as fast-food or supermarket names.
- Category matches help natural queries such as **where can I find a pharmacy**.
- Admin context helps when the query includes a place name.
- The current map view can help nearby POIs, but a named place in the query can override the current view.
- A selected result is highlighted differently in the browser so the clicked ranked result is not confused with the other markers.

The ranking does not invent a place. A brand result is returned only if an OSM object in the loaded PBF has matching local data.

25.20 Natural-language search path

Natural-language mode is used only when the checkbox is enabled. It exists for sentences such as **where can I find burger**, **wo finde ich eine Apotheke**, or **find McDonalds near Stuttgart**.

1. The browser sends the raw query, map center, and viewport to `/api/natural-geocode`.
2. The backend first runs deterministic guards for query shapes that should not need an LLM, such as short address-like queries.
3. If Ollama is enabled and reachable, the backend asks for strict JSON intent, not free prose.
4. The backend asks Ollama to verify the generated JSON against the original user query.
5. The backend repeats verification a second time.
6. The backend validates the JSON schema and allowed intent names.
7. The backend corrects or rejects unsafe/unsupported fields.
8. The backend executes the intent using only local OSM indexes and spatial grids.

Ollama is therefore not the geocoder. It is a parser for messy human wording. The actual results still come from the local PBF snapshot.

25.21 Natural address guard

Some address-like queries are short and should not become brand or POI searches. The backend detects this shape before the named-POI path.

- The query has one number token.
- It has two to four text tokens.
- It has no postcode.
- It looks like an address phrase, not a product sentence.

The backend first tries the full address. If the exact house point is missing, it removes the house number and tries the street-level address. The response marks whether the origin came from **address** or **address_street**.

25.22 Product and service concept handling

The program has local concept knowledge for broad product and service requests. The goal is not to maintain a giant table of every possible product. The goal is to map common product families to OSM categories and tags that can exist in the PBF.

Examples:

- A medicine-like query can map to pharmacies or chemists.
- A grocery-like query can map to supermarkets, convenience shops, or relevant retail categories.
- A car-brand query can use brand/name/operator text and shop/service categories.
- A fast-food brand query can use POI name, brand, operator, and restaurant/fast-food category.

The backend still validates against local data. If the local PBF has no matching POI, the result should be empty or weaker; the program should not create a fake shop.

25.23 Browser map behavior

The browser is a client for the local HTTP server.

- It requests only the current viewport instead of downloading all Europe objects.
- Orange markers show house/address points.
- Blue lines show streets.
- Green polygons show admin areas.
- Purple markers show forward-geocoder result POIs/admin/street/house candidates.
- Clicking a result pans or highlights that specific ranked result.
- The reverse-geocoder checkbox controls whether map clicks trigger reverse lookup.
- The Ollama button tries to start local Ollama service when it is not already reachable.

25.24 Viewport loading

Viewport loading protects the browser from receiving millions of records at once.

1. Leaflet reports the current map bounds.
2. The frontend calls viewport endpoints with min/max latitude and longitude.
3. The server uses spatial indexes to find objects intersecting the bounding box.
4. The server returns only a limited JSON slice for that view.
5. When the user pans or zooms, the browser requests a new slice.

This is separate from geocoder ranking. The search index can search the full loaded Europe snapshot, while map drawing remains viewport-limited.

25.25 Binary snapshot in detail

The binary snapshot stores the compact dataset and forward index so the user does not need to parse the PBF every time.

1. Write a magic header and version information.
2. Write selected language.

3. Write the string table.
4. Write records for houses, streets, admin areas, POIs, and links.
5. Write shared geometry arrays.
6. Write admin ring metadata, including outer and inner rings.
7. Write extraction statistics and timings.
8. Write the forward geocode index.

Old snapshots without the embedded forward index are rejected. This avoids silently rebuilding a huge forward index at server startup and avoids confusing runtime behavior.

25.26 Server startup after loading a snapshot

After `-load-binary`, the server still builds spatial grids. These grids are runtime structures used for fast viewport requests, reverse geocoding, and nearest-POI search. They are smaller and simpler than the full parser state, but they are still needed because they depend on how the HTTP server serves spatial requests.

The forward geocode index is already in the snapshot. That is why normal search does not need to rebuild the forward index at server start.

25.27 Progress bars, timings, and metrics

Progress bars measure byte position in the PBF where libosmium exposes enough information. ETA is shown only after enough progress history exists. At the very beginning it can be unknown because there is not enough speed history to predict the remaining time.

The final summary reports:

- scanned nodes, ways, and relations
- extracted houses, streets, POIs, and admin areas
- missing address fields
- admin links and parent links
- phase timings
- input throughput
- compact dataset size
- peak RSS memory

25.28 Memory reduction rules

The implementation keeps memory lower by removing parser-only state before expensive later phases.

- Strings are interned once, then the hash-map index is released before admin attribution.
- Relation way geometry uses flat coordinate storage and spans instead of many separate vectors.
- Needed-node found flags are packed more tightly.
- Forward posting lists are moved from temporary maps instead of copied.
- Vocabulary is built from sorted posting lists instead of a large temporary hash set.
- Geometry offsets use checked 32-bit casts so overflow becomes a clear error instead of silent corruption.

These changes reduce duplicate storage. They do not remove the real map objects that are required for the implemented features.

26 Live verification summary

The running Europe server was checked with:

- `stuttgart` from Barcelona: first result is admin area Stuttgart
- `munich`: first result is admin area Muenchen
- `muenchen`: first result is admin area Muenchen
- `Aalen Bahnhofstrasse 10`: address result
- `Stuttgart Burger King`: Burger King POI results
- reverse lookup in Stuttgart: nearby street/admin result
- reverse lookup in Kiel: nearby street/admin result
- `Kaistrasse 5 Kiel` with LLM mode: address-shaped query, street-level result when house number is absent

27 Commands

27.1 Build

```
cd /home/ganesh/osm-task1
cmake -S . -B /home/ganesh/osm-task1-build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build /home/ganesh/osm-task1-build
```

27.2 Parse Europe

```
cd /home/ganesh/osm-task1
/home/ganesh/osm-task1-build/osm_parser \
  --parse /home/ganesh/osm-task1/europe-latest.osm.pbf \
  --low-memory \
  --connect-streets \
  --save-binary /home/ganesh/europe-geocoder-v007.bin \
  --pbf-threads 24
```

27.3 Load and serve

```
cd /home/ganesh/osm-task1
OSM_OLLAMA_MODEL=qwen2.5:3b \
OSM_OLLAMA_TIMEOUT_SECONDS=45 \
/home/ganesh/osm-task1-build/osm_parser \
  --load-binary /home/ganesh/europe-geocoder-v007.bin \
  --server 8080
```

27.4 Open browser

`http://localhost:8080`

28 Closing mental model

The project is best understood as four layers:

1. **Extraction layer:** read PBF, keep selected objects, skip irrelevant raw data.
2. **Geometry layer:** build street lines, admin polygons, holes, bboxes, and admin links.
3. **Index layer:** build text indexes, suffix index, fuzzy expansion support, and spatial grids.
4. **Application layer:** serve map tiles overlay data, reverse geocode clicks, forward geocode text, and optionally parse natural sentences through local Ollama.

Every visible result comes from the loaded PBF-derived dataset. The frontend, ranking system, and LLM mode are ways to ask the local dataset better questions; they are not external geocoding sources.